

一、实验目标与内容

- 创建一个进程并成功运行
- 实现时钟中断，通过时钟中断内核可以再次获得执行权
- 实现进程调度，创建两个进程，并且通过时钟中断切换进程执行 在Lab3 中将运行一个用户模式的进程。本实验需要使用数据结构进程控制块Env来跟踪用户进程，并建立一个简单的用户进程，加载一个程序镜像到指定的内存空间，然后让它运行起来。同时，实验实现的MIPS 内核具有处理异常的能力。

二、Thinking

Thinking 3.1:

- `e->env_pgdir[PDX(UVPT)]=PADDR(e->env_pgdir)|PTE_V` PADDR(e->env_pgdir)得到页目录的物理地址，并且有效位置为1，把这个值放到UVPT这个地址对应的页目录索引在env_pgdir中对应的表项里。其实就是在做自映射，把页目录映射到UVPT这个基址上。

Thinking 3.2:

- `maper`中的第一个参数`data`就是传入`elf_load_seg`的`e`参数，得到`env->env_pgdir,env->env_asid`告知`page_insert`地址空间。所以是必要的。

Thinking 3.3:

- `ph`是程序头里面包含了需要加载的页面的大小权限等各种信息，`binary+ph->offset`是文件内要加在的信息的起点，`e`传递地址空间信息，`map`函数用来实现整个过程。

Thinking 3.4

- 是虚拟地址，因为这里存储的是程序入口，`map`时已经放入内存里了，所以是虚拟地址，运行的时候根据该虚拟地址找到入口。

Thinking 3.5

- 在`genex.S`里实现。

Thinking 3.6

- 进入异常时关闭,在 `entry.S` 的 `exc_gen_entry`:

```
mfc0 t0, CP0_STATUS
and t0, t0, ~(STATUS_UM | STATUS_EXL | STATUS_IE)
mtc0 t0, CP0_STATUS
```

- 这里把 `STATUS_IE` 清掉了，所以全局中断关闭。
- 调度并返回前开,在 `env_asm.S` 的 `env_pop_tf`:

```
RESET_KCLOCK
j ret_from_exception
```

- 然后 ret_from_exception 里 RESTORE_SOME + jr EPC + rfe。
- 根据 CP0_STATUS 位选择是否开启。

Thinking 3.7

- 时钟中断进入异常入口 genex.S, 防止被再次中断关闭中断, 然后进入 traps.c, 根据 CAUSE 字段分发异常, 分发给 handle_init, 然后进入 schedule 选好下一个切换的进程, env_run 运行进程, 返回 cp0_rpc.

三、难点分析

- 1. 编写 schedule 函数, 注意 best->env_* 等内容不能初始为 NULL, 或者进行这一步时 best 不能为 NULL, 不然会报错 address too low.
- 2. 添加新的异常处理, 完善整体的异常处理, 主要是 entry.S, genex.S, traps.c 这三个文件, 函数的实现可以写在 genex.S 里也可以在 trap.c 里直接写出, 理解并使用宏定义对各种情况进行分发补充 traps.c

四、实验体会

- 1. 在线考试被 least 不能为 NULL 卡了好久, 虽然课下练习时没有出现这个错误, 还是要注意, 不然 tlb 会报错。
- 2. 这次通过 entry.S -> genex.S -> schedule -> env_run -> env_pop_tf 这条链, 学习了 CPU 如何从用户态陷入内核、保存现场、选择下一个进程、再恢复现场返回用户态。对“上下文切换”的理解也更具体了: 本质是寄存器现场、地址空间 (页目录/ASID) 和执行入口 (EPC) 的切换。IE/EXL/UM 位的开关时机、COUNT/COMPARE 的重置、ACK 中断源等都直接影响系统是否能稳定抢占。整体来看, 异常是控制权回收机制, 调度是控制权再分配机制。二者配合后, 系统才能实现多进程并发与公平运行。